

## Heat

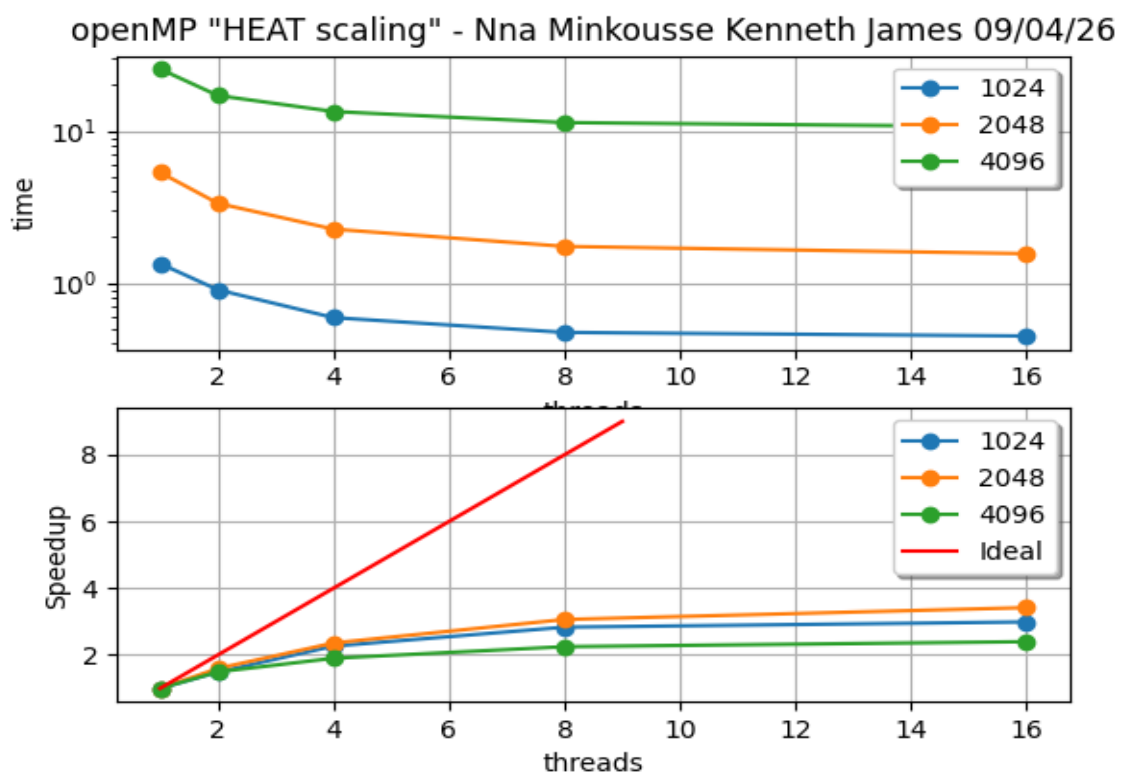
In this program I added a directive to make the program concurrent with thread at the level of the Jacobi function to optimize the speedup of the heat-map program. This is the screenshot of the modification I did.

```
void Jacobi_Iterator_CPU(float * __restrict T, float * __restrict T_new, const int NX, const int NY)
{
    int i,j;

    // --- Only update "interior" (not boundary) node points
    // #pragma omp parallel for private (i) //every thread will have its own value of i
    #pragma omp parallel for collapse(2) // also we can do this like this that is still a solution
    for(j=1; j<NY-1; j++)
        for(i=1; i<NX-1; i++) {
            float T_E = T[(i+1) + NX*j];
            float T_W = T[(i-1) + NX*j];
            float T_N = T[i + NX*(j+1)];
            float T_S = T[i + NX*(j-1)];
            T_new[NX*j + i] = 0.25*(T_E + T_W + T_N + T_S);
        }
}
```

this is the output of the result obtain at the end of the compilation:

→ We can see that as the array becomes big it takes more time to resolve the problem.  
→ About the speedup we also observe that as from 8 threads it becomes liner the speedup



Also looking at the output of the python script we can see the time it takes to compute just on the Jacobi function (tj) and the total time of the whole program. With constant columns and rows. But increasing number of threads that help reduce the time of work.

|     | nt | r    | c    | iter | t     | tj    | hostname |
|-----|----|------|------|------|-------|-------|----------|
| OMP | 1  | 2048 | 2048 | 500  | 5.338 | 4.167 | wn99     |
| OMP | 2  | 2048 | 2048 | 500  | 3.345 | 2.155 | wn99     |
| OMP | 4  | 2048 | 2048 | 500  | 2.263 | 1.088 | wn99     |
| OMP | 8  | 2048 | 2048 | 500  | 1.742 | 0.582 | wn99     |
| OMP | 16 | 2048 | 2048 | 500  | 1.561 | 0.406 | wn99     |

## factorize

This is how I modified the code:

```
double t1, t2;          // Time variables (start and end)
int found_local;        // Local result for each thread

#pragma omp parallel for schedule(dynamic,1)
    private(t1, t2, found_local) shared(found)
for (block_idx = block_addr_size - 1; block_idx ≥ 0; block_idx--)

    // --- Synchronize 'found' across all threads ---
    #pragma omp flush(found)

    // If another thread already found the solution, skip work
    if (found) continue;

// --- Safe printing (avoid mixed output) ---
#pragma omp critical
    printf("thread %d processing block %d\n", omp_get_thread_num(), block_idx);
    t1 = omp_get_wtime();
// --- Do the actual computation ---
    found_local = block_factorize(block_idx);
    t2 = omp_get_wtime();
// --- If this thread found the result ---
    if (found_local) {

        // Safely update shared variable
        #pragma omp critical
        {
            found = 1;
        }

        #pragma omp flush(found)

// --- Safe printing of results ---
#pragma omp critical
        printf("thread %d processed block %d in %.1f sec. result %d\n", omp_get_thread_num(), block_idx, t2 - t1, found_local);
    }
}
```

I preferred dynamic in the schedule because it will reduce the overhead that is when one thread finishes its work in its block let it take goes in another block to look for the prime number divisible by the modulus.

Also **found** is a shared variable between threads so when one thread changes the value of the variable **found** it notifies all other threads that the value have change and the value of the other prime number divisible by the modulus is found.

→ So I can say that adding dynamic, 1 on the schedule helps the threads to be synchronize between themself.